DATA PREPARATION FOR MACHINE LEARNING AND AI · TEXT DATA MODULE

Classical text pre-processing, and when it hurts

Text data, class 1 of 2. From bytes to words to vectors: Unicode, tokenization, normalization, bag-of-words, classical augmentation, and the languages that never had spaces.

Asst. Prof. Thanapong Intharah, PhD

Visual Intelligence Laboratory · Khon Kaen University

A curated compilation of openly published university teaching materials. Every section names its source; original slides are shown unmodified.

Today, in three hours

0:00–0:25	**A0 · Why preprocess, why count words.** Models eat counts: bag of words, n-gram zeros, Zipf and Heaps on a real novel, one name as 22 strings.	SLP3 NB + LM slides + authored
0:25–0:45	**A1 · What is a word?** Types and tokens, languages without spaces, Herdan's law, morphemes, corpora.	SLP3 slides (Jurafsky & Martin)
0:45–1:00	**A2 · Text is bytes.** Unicode, UTF-8, normalization forms, mojibake, extraction from PDFs and HTML.	SLP3 + authored
1:00–1:15	**A3 · Tokenization, rule-based.** Regex essentials, Penn Treebank conventions, sentence segmentation, spaCy.	SLP3 + authored
1:15–1:25	*Break*	
1:25–1:40	**A4 · Normalization.** Case, stemming, lemmatization, stop words, and when this now hurts.	SLP3 + authored
1:40–2:00	**A5 · Thai.** Segmentation without spaces, PyThaiNLP live, ambiguity, Thai normalization.	authored (live demos)
2:00–2:20	**B1 · From tokens to vectors.** Bag of words, tf-idf, worked example, scikit-learn.	SLP3 IR slides + authored
2:20–2:35	**B2 · Classical augmentation.** EDA live, back-translation, when small data justifies it.	authored
2:35–3:00	**C · Clinical text, classical era.** Telegraphic notes, term spotting, NegEx, UMLS, regex de-identification.	MIT OCW 6.S897 + authored

This deck is a guided tour of open course materials

- **Each dark divider names the source deck** that the following slides come from: course, lecture, authors, institution, and a link to the complete original.
- Source slides are shown **exactly as published, unmodified**; their own slide numbers are kept so you can find the full context.
- The MIT OpenCourseWare section is **CC BY-NC-SA 4.0**. The Jurafsky and Martin (SLP3) slides are posted freely for teaching and are shown with attribution; please **do not re-post or adapt** those sections without checking the authors' terms.
- Slides marked AUTHORED in the footer were written for this course. Their demos were **actually run** (PyThaiNLP, NLTK, scikit-learn, ftfy); outputs are real, not typed.

Full source list, links, and licenses: last two slides of this deck.

PART A0 · 0:00–0:25

Why preprocess text at all? Why count words?

Because every classical text model is a machine that eats counts: a classifier sees a bag of word counts, a language model sees counts of word sequences, a search engine sees weighted counts. What you decide to count is the only world the model ever sees.

SOURCE DECKS

Naive Bayes and Text Classification (January 2025) and **N-gram Language Models** (January 2025), Speech and Language Processing, 3rd ed. draft slides · Dan Jurafsky and James H. Martin; framed by authored slides with live corpus statistics.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

Every text model consumes counts of something

MODEL FAMILY	WHAT IT ACTUALLY RECEIVES	SO PREPROCESSING DECIDES
Naive Bayes, logistic regression, SVM on text	a bag of word counts per document	what a "word" is, whether "Pain" and "pain," are the same word, whether "not" survives
n-gram language model	counts of word sequences	the vocabulary, hence which sequences are ever seen and which are zeros
tf-idf retrieval, BM25	weighted counts of terms per document	stemming, stop words, and what counts as a document
word2vec, GloVe	co-occurrence counts in a window	tokenization and case, which decide which contexts are shared
transformer language model	a sequence of token ids from a fixed vocabulary	the tokenizer (class 2), and everything that shaped the corpus it was trained on

- None of these models sees letters, meaning, or grammar. They see **the units you chose to count, in the quantities your pipeline produced**.
- Hence the two questions of this class: **what should count as a word** (tokenization, normalization) and **how many of them are there** (vocabulary, frequency, sparsity).

Positive or negative movie review?

-  unbelievably disappointing
-  Full of zany characters and richly applied satire, and some great plot twists
-  this is the greatest screwball comedy ever filmed
-  It was pathetic. The worst part about it was the boxing scenes.

Text Classification

Assigning subject categories, topics, or genres

Spam detection

Authorship identification (who wrote this?)

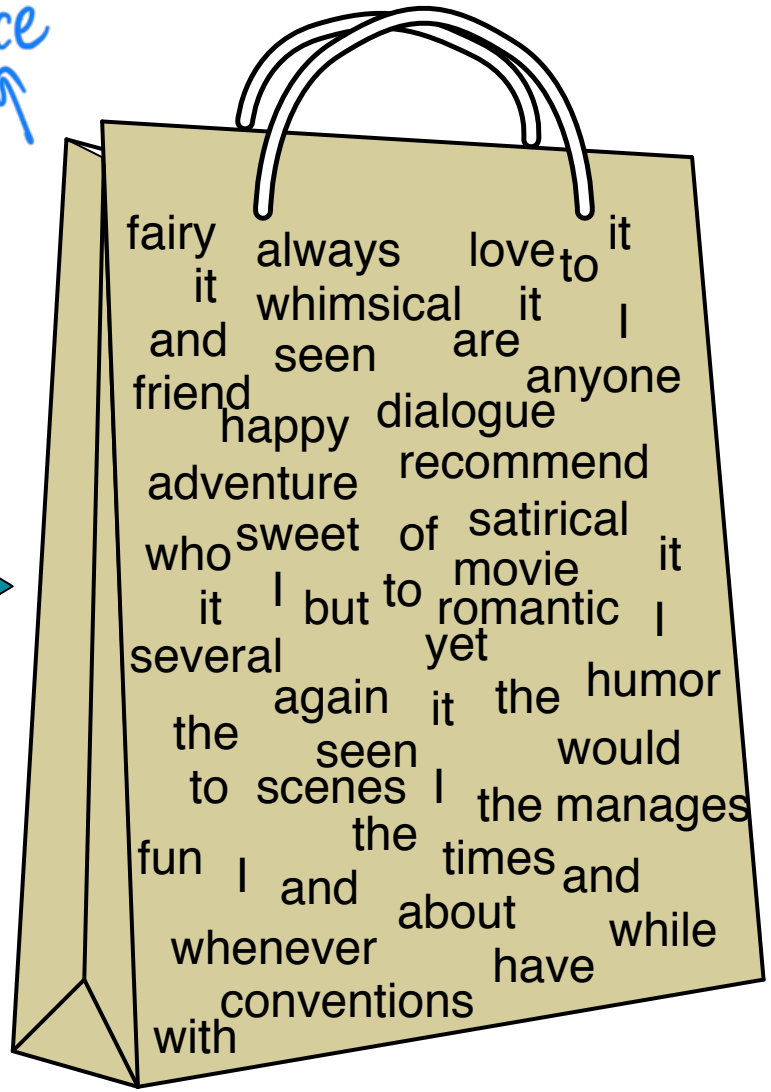
Language Identification (is this Portuguese?)

Sentiment analysis → ดูเหมือนว่าดีหรือไม่ดี

Language modeling (what next word does this context expect)

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

The bag of words representation

$\gamma($

seen	2
sweet	1
whimsical	1
recommend	1
happy	1
...	...

$) = c$



Unknown words

What about unknown words

- that appear in our test data
- but not in our training data or vocabulary?

We **ignore** them

- Remove them from the test document!
- Pretend they weren't there!
- Don't include any probability for them at all!

Why don't we build an unknown word model?

- It doesn't help: knowing which class has more unknown words is not generally helpful!

Stop words

↳ คำที่ความหมายไม่ค่อยชัดเจน แต่ไม่สามารถสื่อความหมายถึงตัวมันเองได้

Some systems ignore stop words

- **Stop words:** very frequent words like *the* and *a*.
  - Sort the vocabulary by word frequency in training set
  - Call the top 10 or 50 words the **stopword list**.
  - Remove all stop words from both training and test sets
    - As if they were never there!

But removing stop words doesn't usually help

- So in practice most NB algorithms use **all** words and **don't** use stopword lists

Sentiment Classification: Dealing with Negation

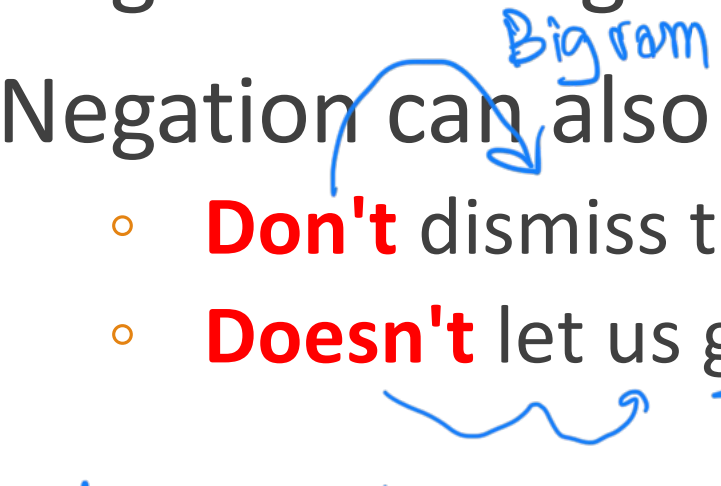
I really like this movie

I really **don't** like this movie

Negation changes the meaning of "like" to negative.

Negation can also change negative to positive-ish

- **Don't** dismiss this film
- **Doesn't** let us get bored



Sentiment Classification: Dealing with Negation

Das, Sanjiv and Mike Chen. 2001. Yahoo! for Amazon: Extracting market sentiment from stock message boards. In Proceedings of the Asia Pacific Finance Association Annual Conference (APFA).
Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. EMNLP-2002, 79—86.

Simple baseline method:

Add NOT_ to every word between negation and following punctuation:

didn't like this movie , but I



didn't NOT_like NOT_this NOT_movie but I

Predicting words

The water of Walden Pond is beautifully ...

blue
green
clear

*refrigerator
*that



Language Models

A language model is a machine learning model that predicts upcoming words.

- More formally:
  - An LM assigns a probability to each potential next word
    - = gives a probability distribution over possible next words
  - An LM assigns a probability to a whole sentence

Two paradigms

- N-gram language models (this lecture)
- Large language models (LLMs, neural models, future lectures)

Shakespeare as corpus

N=884,647 tokens, V=29,066

Shakespeare produced 300,000 bigram types out of V^2= 844 million possible bigrams.

- So 99.96% of the possible bigrams were never seen (have zero entries in the table)
- That sparsity is even worse for 4-grams, explaining why our sampling generated actual Shakespeare.

The Wall Street Journal is not Shakespeare

1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Choosing training data

If task-specific, use a training corpus that has a similar genre to your task.

- If legal or medical, need lots of special-purpose documents

Make sure to cover different kinds of dialects and speaker/authors.

- Example: *African-American Vernacular English (AAVE)*
  - One of many varieties that can be used by African Americans and others
  - Can include the auxiliary verb **finna** that marks immediate future tense:
  - "My phone finna die"

Zeros

Training set:

… ate lunch
… ate dinner
… ate a
… ate the

P(“breakfast” | ate) = 0

- Test set
  … ate lunch
  … ate breakfast

Zero probability bigrams

Bigrams with zero probability

- Will hurt our performance for texts where those words appear!
- And mean that we will assign 0 probability to the test set!

And hence we cannot compute perplexity (can't divide by 0)!

The problem of sparsity

Maximum likelihood estimation has a problem

- Sparsity! Our finite training corpus won't have some perfectly fine sequences
  - Perhaps it has "ruby" and "slippers"
  - But happens not to have "ruby slippers"

Training sets and test sets

We train parameters of our model on a **training set**.

We test the model's performance on data we haven't seen.

- A **test set** is an unseen dataset; different from training set.
  - Intuition: we want to measure generalization to unseen data
- An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.

Training on the test set

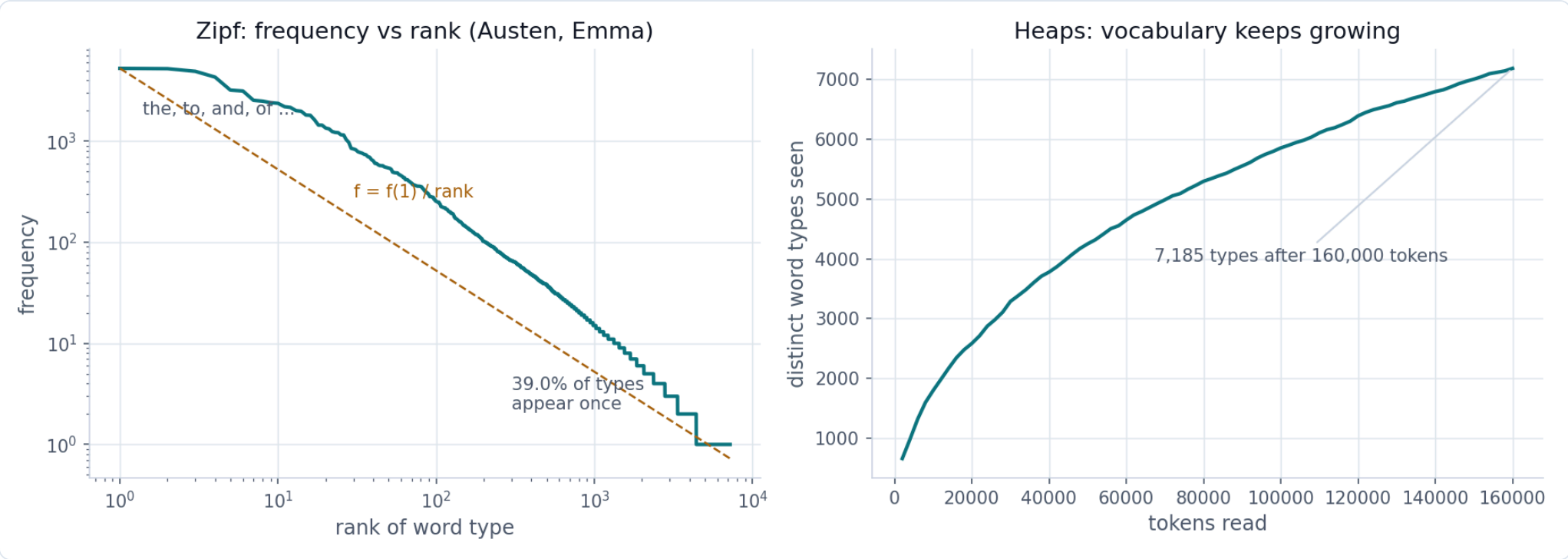
We can't allow test sentences into the training set

- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called "Training on the test set"

Bad science!

Why word counts are never "enough data"



- Jane Austen's *Emma*, 161,096 word tokens, 7,225 word types (lowercased letters only): the **top 10 types cover 22.2%** of all tokens, the top 100 cover 59.6%, and **39% of types occur exactly once**.
- Zipf's law (frequency falls as 1/rank) and Heaps' law (the vocabulary never stops growing) mean every text dataset is mostly rare events: **sparsity, zeros, and unknown words are guaranteed**, which is what normalization, smoothing, and later subword tokenization exist to manage.

One novel, four vocabularies

HOW WE DEFINED A "WORD"	TYPES |V|	VS. PREVIOUS
split on whitespace, nothing else	17,409	
plus lowercasing	16,945	−3%
letters only, punctuation removed	7,225	−57%
plus Porter stemming	4,611	−36%

With whitespace splitting alone, the heroine's name appears as 22 different strings: `Emma` `Emma,` `Emma.` `"Emma` `Emma!"` `(Emma` `Emma;` `Emma?"` `...`

- A naive pipeline spends **most of its vocabulary on punctuation variants** of the same word; every count is split across them, every estimate gets noisier.
- Each normalization step **shrinks the vocabulary and merges evidence**, which is exactly why classical NLP did it.
- Each step also **destroys a distinction**: "US" versus "us", "Emma?" versus "Emma.", "caring" versus "cares". Part A4 asks when that trade stops paying, and it does stop with modern models.
- Counting first turns the debate into a measurement: **how many types, how many singletons, what the top ten are**. Do this before any model.

Five numbers and one question for every text dataset

- **1. Tokens (N)**: how much text there is; decides whether an n-gram model, a fine-tuned classifier, or only a pretrained model is realistic.
- **2. Types (|V|) and the type/token ratio**: vocabulary size sets the feature space, the embedding table, and how many parameters counts must feed.
- **3. Singletons and the frequency curve**: the share of hapax types predicts the unknown-word rate on new data (Austen: 39%).
- **4. Document length distribution**: median and tail decide truncation, padding, and chunking later.
- **5. Composition**: classes, genres, dialects, authors, dates, languages; the SLP3 corpus datasheet questions in numbers.

And the question: does the test set come from the same world as the training set? Shakespeare is not the Wall Street Journal; patient A's notes are not patient B's.

- "Training on the test set" (the LM slides) is the text-world name for the leakage taxonomy of course section 1.11.
- Every later part of today changes at least one of these numbers; write them down before and after each step, and you have your datasheet.

PART A1 · 0:25–0:45

What is a word? Harder than it looks

Types versus tokens, vocabulary that grows without bound, languages that never used spaces, and the corpus questions (who wrote this, when, for whom) that decide what your model will learn.

SOURCE DECK

Words and Tokens (chapter 2 slides, January 2026), Speech and Language Processing, 3rd ed. draft · Dan Jurafsky and James H. Martin, Stanford University / University of Colorado.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

How many words in a sentence?

```
They picnicked by the pool, then
lay back on the grass and looked at
the stars.
```

Type: an element of the vocabulary V

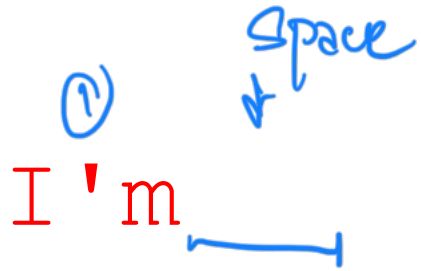
- The number of types is the vocabulary size |V|

Instance: an instance of that type in running text.

- 14 types and 16 instances (if we ignore punctuation).

More questions: Are `They` and `they` the same word?

How many words in a sentence?



Orthographically one word (in the English writing system)



But **grammatically** two words:

1. the subject pronoun I
2. the verb 'm, short for am

How many words in a sentence?

Not every written language uses spaces!!

Chinese, Japanese and Thai don't!

sparse → กระจาย Dimention มิติ เยอะ เกิน ไม่ดี

How to choose tokens in Chinese

Chinese words are composed of characters called "**hanzi" (汉字, hànzì)** (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.

Tokenization across languages

- So in Chinese we use characters (zi) as tokens
  - But that doesn't work for, e.g., Thai and Japanese
  - These differences make it hard to use words as tokens
- And there's another reason why we don't use words as tokens!

There are simply too many words!

	Types = |V|	Instances = N
Shakespeare	31 thousand	884,000
Brown Corpus	38 thousand	1 million
Switchboard conversations	20 thousand	2.4 million
COCA	2 million	440 million
Google N-grams	13+ million	1 trillion

The bigger the corpus, the more word types!

There are simply too many words!

N = number of instances

$|V|$ = number of types in vocabulary V

Heaps Law = Herdan's Law

$$|V| = kN^{\beta}$$ ← Roughly 0.5

Vocab size for a text goes up with the square root of its length in words

Two kinds of words

Function words

- of, the, is, and, una, 是,

Content words

- mango, braise, snowy, feliz, 北京

Why is too many words a problem?

No matter how big our vocabulary

There will always be words we missed!

We will always have unknown words!

Words and Subwords

Because of these three problems:

1. Many languages don't have orthographic words
2. Defining words post-hoc is challenging
3. The number of words grows without bound

NLP systems don't use words, but smaller **subwords**

Words have parts

Morpheme: a minimal meaning-bearing unit in a language.

`fox`: one morpheme

`cats`: two morphemes `cat` and `-s`

Morphology: the study of morphemes

Types of morphemes

root: central morpheme of the word
- supplying the main meaning

affix: adding additional meanings

worked
- *root* *work*
- *affix* *-ed*

glasses
- *root* *glass*
- *affix* *-es*

Types of affixes

Inflectional morphemes

- grammatical morphemes
- often syntactic role like agreement
  - `-ed` past tense on verbs
  - `-s/-es` plural on nouns

Derivational morphemes

- more idiosyncratic in application and meaning
- often change grammatical class
  - *`care`* (noun)
    - + *`-full`* → `careful` (*adjective*)
    - + *`-ly`* → *`carefully`* (*adverb*)

Morphological Typology

Dimensions along which languages vary

Two are salient for tokenization:

1. number of morphemes per word
2. how easy it is to segment the morphemes

How easily segmentable

Agglutinative languages like Turkish
- Very clean boundaries between morphemes

Fusion languages
- a single affix may conflate multiple morphemes,

- Russian `-om` in `stolom` (table-SG-INSTR- DECL1)
  - instrumental, singular, and first declension.
- English `-s` in "`She reads the article`"
  - Means both "third person" and "present tense"

These are tendencies rather than absolutes

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer(s),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Corpora vary along dimensions like

Language: 7097 languages in the world

It's important to test algorithms on multiple languages

What may work for one may not work for another

Corpora vary along dimensions like

Variety, like African American English varieties

- AAE Twitter posts might include forms like "*iont" (I don't)*

Genre: newswire, fiction, scientific articles, Wikipedia

Author Demographics: writer's age, gender, ethnicity, socio-economic status

Code Switching

Speakers use multiple languages in the same utterance

This is very common around the world

Especially in spoken language and related genres like texting and social media

Code Switching: Spanish/English

Por primera vez veo a @username actually being hateful! It was beautiful:)

[For the first time I get to see @username actually being hateful! it was beautiful:)]

Corpus **datasheets**

Gebru et al (2020), Bender and Friedman (2018)

Motivation:

- Why was the corpus collected?
- By whom?
- Who funded it?

Situation: In what situation was the text written?

Collection process: How was it sampled? Was there consent? Pre-processing?

+Annotation process, variety, demographics, etc.

PART A2 · 0:45–1:00

Text is bytes: Unicode, UTF-8, and mojibake

Every "string" your pipeline touches was decoded from bytes under some encoding assumption. Get it wrong and you preprocess garbage confidently.

SOURCE DECK

Words and Tokens, Unicode section (January 2026), Speech and Language Processing, 3rd ed. draft · Dan Jurafsky and James H. Martin; followed by authored normalization and extraction slides.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

Words and Tokens

Unicode

Unicode

a method for representing text written using

- any character (more than 150,000!)
- in any script (168 to date!)
- of the languages of the world
  - Chinese, Arabic, Hindi, Cherokee, Ethiopic, Khmer, N'Ko,…
  - dead ones like Sumerian cuneiform
  - invented ones like Klingon
  - plus emojis, currency symbols, etc.

ASCII: Some history for English

1960s American Standard Code for Information Interchange

- 1 byte per character
  - In principle 256 characters
  - But high bit set to 0
  - So 7 bits = 128
  - However only 95 used
- The rest were for teletypes



Code Points

Unicode assigns a unique ID, a **code point,** to each of its 150,000 characters

1.1 million possible code points

- 0 – 0x10FFFF

Written in hex, with prefix "U+"

- a is U+0061 which = 0x0061

First 127 code points = ASCII

- For backwards compatibility

Encodings and UTF-8

We don't stick code points directly in files

We store **encodings** of chars.

The most popular encoding is UTF-8

Most of the web is stored in UTF-8

Encodings

`hello` has these 5 code points:

U+0068 U+0065 U+006C U+006C U+006F

How to write in a file?

There are more than 1 million code points

So would need 4 bytes (or 3 but 3 is inconvenient):

00 00 00 68 00 00 00 65 00 00 00 6C 00 00 00 6C 00 00 00 6F

But that would make files very long!

- Also zeros are bad (since mean "end of string" in ASCII)

Instead: Variable Length Encoding

UTF-8 (Unicode Transformation Format 8)

For the first 127 code points, same as ASCII

UTF-8 encoding of `hello` is :

- 68 65 6C 6C 6F

Code points ≥128 are encoded as a sequence of 2, 3, or 4 bytes

- In range 128 - 255, so won't be confused with ASCII
- First few bits say if its 2-byte, 3-byte, or 4-byte

UTF-8 encoding

The first 127 characters (ASCII) map to 1 byte

Most remaining characters in European, Middle Eastern, and African scripts map to 2 bytes

Most Chinese, Japanese, and Korean characters map to 3 bytes

Rarer CJKV characters, emojis/symbols map to 4 bytes.

UTF-8 encoding

Efficient: fewer bytes for common characters,

Doesn't use **zero bytes** (except for NULL character U+0000),

Backwards compatible with **ASCII**,

Self-synchronizing,

- If a file is corrupted, the nearest character boundary is always findable by moving only up to 3 bytes

UTF-8 and Python 3

Python 3 strings stored internally as Unicode

- each string a sequence of Unicode code points
- string functions, regex apply natively to code points.
  - **len() returns string length in code points, not bytes**

Files need to be encoded/decoded when written or read

- Every file is stored in some encoding
- ***No such thing as a text file without an encoding***
  - If it's not UTF-8 it's something older like ASCII or iso_8859_1

Two strings that look identical and compare unequal

```
import unicodedata, ftfy
s1 = "café"   # e + combining acute, 5 code points
s2 = "café"    # precomposed e-acute, 4 code points
s1 == s2                                    # False
unicodedata.normalize("NFC", s1) == s2       # True

# NFKC also folds compatibility characters
unicodedata.normalize("NFKC", "financial")  #
financial
unicodedata.normalize("NFKC", "H e l l o  2 0 2 6")
# Hello 2026

# ftfy repairs double-decoded text
ftfy.fix_text("reported â€œchest painâ€\x9d, Ã©tat
fÃ©brile")
# reported "chest pain", état fébrile
```

- **NFC** for storage and matching; **NFKC** when you also want ligatures, full-width forms, and circled digits folded (careful: it is lossy by design).
- **Mojibake** (UTF-8 read as Latin-1) is the text equivalent of the BGR/RGB bug from the image classes: nothing crashes, every downstream count is wrong.
- Decide the normalization form **once**, apply it at ingestion, and log it in the datasheet; a tokenizer trained on NFC text meets NFD input as unknown characters.

Extraction is the first preprocessing step nobody reports

```
# PDFs: layout-aware text, not just characters
import pymupdf                          # (fitz)
doc = pymupdf.open("report.pdf")
text = "\n".join(p.get_text("text") for p in doc)

# HTML: strip boilerplate, keep the article
import trafilatura
main = trafilatura.extract(html, include_tables=True)

# Scans: OCR, then the same normalization path
import pytesseract
raw = pytesseract.image_to_string(img, lang="tha+eng")
```

- **PDF text order is not reading order**: two-column papers interleave, tables shatter, hyphen-ated line breaks split words. Inspect a sample before trusting a million pages.
- **HTML boilerplate** (menus, cookie banners, "related articles") is the single largest source of duplicated text in web corpora; strip it before deduplication (class 2).
- **OCR errors are systematic**, not random: `rn` becomes `m`, Thai tone marks drift. Keep the page image so errors can be audited.
- Record the extractor and version in the datasheet. Swapping PDF libraries changes the corpus.

PART A3 · 1:00–1:15

Tokenization by rules: regex, Treebank, sentences

Before learned tokenizers there were conventions. They still run inside every pipeline: pre-tokenization for BPE, sentence splitting for RAG, regex for cleaning.

SOURCE DECK

Words and Tokens, regular expressions and rule-based tokenization sections (January 2026), Speech and Language Processing, 3rd ed. draft · Dan Jurafsky and James H. Martin.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

Regular expressions are used everywhere

- A formal language for specifying text strings
- Part of every text processing task
  - Useful pre-processing or text formatting step, for example for BPE pre-tokenization
- Also necessary for data analysis of text
- A widely used tool in industry and academics

Regular expressions

The pattern

```
r"Buttercup"
```

matches the substring Buttercup in any string, like the string

```
I'm called little Buttercup
```

The iterative process of writing regex's

Find me all instances of the word “the” in a text.

```
the
```

Misses capitalized examples

```
[tT]he
```

Incorrectly returns `other` or `Theology`

```
\W[tT]he\W
```

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched (The)

   False negatives

2. Matching strings that we should not have matched (there, then, other)

   False positives

Regular expressions play a surprisingly large role

Widely used in both academics and industry

1. Part of most text processing tasks, even for big neural language model pipelines
   - including text formatting and pre-processing
2. Very useful for data analysis of any text data

Regex Substitutions in Python

To change every instance of cherry to apricot in string:

```
re.sub(r"cherry", r"apricot", string)
```

Upper case all examples of a name:

```
re.sub(r"janet", r"Janet", string)
```

Capture group

```
re.sub(r"(\d{2})/(\d{2})/(\d{4})",
r"\2-\1-\3", string)}
```

Parens (and) around the two month digits, the two day digits, and the four year digits,

This stores

- the first 2 digits in group 1,
- the second 2 digits in group 2,
- final digits in group 3.

Then in the *repl* string,

- \1, \2, and \3, refer to the 1st, 2nd, and 3rd registers.

Rule-based tokenization

Although subword tokenization is the norm

Sometimes we need particular tokens

Like for parsing, where the parser needs grammatical words, or social science

Issues for rule-based tokenization

Mostly but not always remove punctuation:

- m.p.h., Ph.D., AT&T, cap'n
- prices ($45.55)
- dates (01/02/06)
- URLs (http://www.stanford.edu)
- hashtags (#nlproc)
- email addresses (someone@cs.colorado.edu)

Numbers are tokenized differently across languages

- English 555,500.50 = French 555 500,50

Multiword expressions (MWE)?

- New York, rock 'n' roll

Penn Treebank Tokenization Standard

Input: "The San Francisco-based restaurant," they said,
"doesn't charge $10".

Output: "␣The␣San␣Francisco-based␣restaurant␣,␣"␣they␣said␣,␣
"␣does␣n't␣charge␣$␣10␣"␣.

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)    # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...   | \w+(?:-\w+)*        # words with optional internal hyphens
...   | \$?\d+(?:\.\d+)?%?  # currency, percentages, e.g. $12.40, 82%
...   | \.\.\.              # ellipsis
...   | [][.,;"'?():_`-]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Sentence Segmentation

!, ? mostly unambiguous but **period** "." is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

spaCy does the whole classical stack in one call

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Dr. Smith didn't see the U.S. patient at 8
a.m.")

for sent in doc.sents:              # sentence
segmentation
    for tok in sent:
        print(tok.text, tok.lemma_, tok.pos_,
              tok.is_stop, tok.is_punct)

# Dr. | did | n't | U.S. | 8 | a.m.  stay intact:
# abbreviations, clitics, times are rule-handled

# NLTK equivalents
from nltk.tokenize import word_tokenize, sent_tokenize
```

- Tokenizer, tagger, lemmatizer, and sentence splitter form a **pipeline** in the literal sense: each stage's errors feed the next.
- Rule sets are **language-specific**: `en_core_web_sm` knows "Dr." and "n't"; Thai needs a different segmenter entirely (part A5).
- Version-pin the model. spaCy and NLTK tokenizers **change between releases**, which silently changes vocabulary counts and every feature built on them.

PART A4 · 1:25–1:40

Normalization: case, stems, lemmas, and stop words

Collapsing surface variation was the point of classical preprocessing. It shrank vocabularies and helped tf-idf. It also throws away signal that modern models exploit.

SOURCE DECK

Basic Text Processing, word normalization section (2023), Speech and Language Processing, 3rd ed. draft · Dan Jurafsky and James H. Martin.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

Word Normalization

Putting words/tokens in a standard format

- U.S.A. or USA
- uhhuh or uh-huh
- Fed or fed
- am, is, be, are

Case folding

Applications like IR: reduce all letters to lower case

- Since users tend to use lower case
- Possible exception: upper case in mid-sentence?
  - e.g., ***General Motors***
  - ***Fed*** vs. ***fed***
  - ***SAIL*** vs. ***sail***

For sentiment analysis, MT, Information extraction

- Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish quiero (‘I want’), quieres (‘you want’)
  → querer ‘want'
- *He is reading detective stories*
  → *He be read detective story*

Lemmatization is done by Morphological Parsing

Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

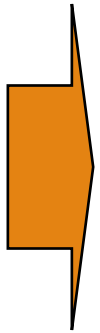
Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*
- Parse Spanish *amaren* ('if in the future they would love') into morpheme *amar* 'to love', and the morphological features *3PL* and *future subjunctive*.

Stemming

Reduce terms to stems, chopping off affixes crudely

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note .

Porter Stemmer

Based on a series of rewrite rules run in series

- A cascade, in which output of each pass fed to next pass

Some sample rules:

$$\begin{aligned}
\text{ATIONAL} &\rightarrow \text{ATE} \quad (\text{e.g., relational} \rightarrow \text{relate}) \\
\text{ING} &\rightarrow \epsilon \quad \text{if stem contains vowel (e.g., motoring} \rightarrow \text{motor)} \\
\text{SSES} &\rightarrow \text{SS} \quad (\text{e.g., grasses} \rightarrow \text{grass})
\end{aligned}$$

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- Uygarlastiramadiklarimizdanmissinizcasina
- `(behaving) as if you are among those whom we could not civilize'
- Uygar `civilized' + las `become'

  + tir `cause' + ama `not able'

  + dik `past' + lar 'plural'

  + imiz 'p1pl' + dan 'abl'

  + mis 'past' + siniz '2pl' + casina 'as if'

Porter versus WordNet on eight words (NLTK, real output)

WORD	PORTER STEM	WORDNET LEMMA
studies	studi	study
studying	studi	study
caring	care	care
was	wa	be
leaves	leav	leave
better	better	better
hospitalization	hospit	hospitalization

Lemmas needed a part-of-speech hint ("was" as a verb); the stemmer needed nothing and produced non-words.

"The patient did not report any chest pain, but was not able to walk."

after NLTK's 198 English stop words: `patient report chest pain able walk`

- **Both negations vanished.** For a clinical classifier this flips the label. Stop-word lists are a choice you must audit, never a default.
- Stemming is fast and crude, lemmatization slower and linguistic; for tf-idf either can help, for transformers neither is applied.

When classical preprocessing now hurts

OPERATION	TF-IDF / LINEAR MODELS	SUBWORD TRANSFORMERS
lowercasing	usually helps	usually harms (cased models learn "US" vs "us")
stop-word removal	often helps	harms: negation, function words carry structure
stemming / lemmatizing	helps sparse counts	harms: subwords already share stems
punctuation stripping	neutral	harms: "?" and "!" are signal
Unicode NFC + mojibake repair	helps	**still helps**
boilerplate removal, dedup	helps	**still helps** (class 2)

- The modern default feeds **near-raw text** to a learned tokenizer; aggressive cleaning destroys information the model would have used (course section 2.1).
- What survives the shift is the **honest** cleaning: encoding repair, extraction quality, deduplication, and consistency between training and deployment.
- Rule of thumb: **match the model**. A pretrained checkpoint expects text shaped like its pretraining corpus, exactly as ImageNet models expect ImageNet statistics.

PART A5 · 1:40–2:00

Thai: tokenizing a language that never had spaces

Every English pipeline assumed whitespace. Thai writes sentences as unbroken runs of characters, so word segmentation is itself a model, with its own errors and ambiguities. Live demos with PyThaiNLP.

AUTHORED

Written for this course; demos run with **PyThaiNLP** (Apache-2.0; Phatthiyaphaibun et al., NLP-OSS 2023) on the built-in dictionary engines.

pythainlp.org · github.com/PyThaiNLP/pythainlp

One sentence, no spaces, 65 characters, 195 bytes

ผู้ป่วยเข้ารับการรักษาในโรงพยาบาลด้วยอาการเจ็บหน้าอกและหายใจลำบาก

"The patient was admitted to the hospital with chest pain and shortness of breath."

- **No word boundaries**: whitespace marks phrase or sentence breaks, not words. `str.split()` returns the whole sentence.
- **Three bytes per character** in UTF-8: โรงพยาบาลศรีนครินทร์ is 20 characters and 60 bytes; "Srinagarind Hospital" is 20 and 20. Byte-level tokenizers start Thai at a 3x disadvantage (class 2).
- **Combining marks**: vowels and tone marks stack above and below consonants; a naive character reversal or truncation breaks syllables.
- **Analytic morphology**: no inflection to stem, so stemming is meaningless; compounds carry the meaning instead (โรง + พยาบาล = building + nurse = hospital).
- **Code switching is the norm** in Thai clinical and technical text: English abbreviations, drug names, and units inside Thai sentences (recall the SLP3 code-switching slides).
- Consequence: for Thai, **segmentation is the preprocessing decision**; everything downstream inherits its errors.

Dictionary-based segmentation in two lines

```
from pythainlp.tokenize import word_tokenize
s = "ผู้ป่วยเข้ารับการรักษาในโรงพยาบาล..."
word_tokenize(s, engine="newmm")   # default
word_tokenize(s, engine="longest")
```

newmm output (13 words):

ผู้ป่วย | เข้า | รับ | การรักษา | ใน | โรงพยาบาล | ด้วย | อาการ | เจ็บ | หน้าอก | และ | หายใจ | ลำบาก

- **newmm**: maximal matching over a dictionary trie with Thai character-cluster rules; fast, deterministic, the sensible default.
- **longest**: greedy longest match; agrees with newmm here, disagrees on ambiguous strings (next slide).
- Neural engines (`deepcut`, `attacut`) trade speed for fewer errors on out-of-dictionary words: names, slang, transliterated drug names.
- **Report the engine and its version** in your methods section: it defines your vocabulary as surely as a tokenizer checkpoint does.

Same characters, two readings, two engines

ขนมอบ

ENGINE	SEGMENTATION	READING
newmm	ขน | มอบ	fur + hand over
longest	ขนม | อบ	pastry + baked (intended)

ตากลม

Both engines return one dictionary word; the alternative reading ตาก + ลม (dry in the wind) is unreachable without context.

- Segmentation errors are **silent**: no exception, a plausible token list, wrong meaning. Only a native speaker or a downstream metric notices.
- Dictionary engines cannot resolve ambiguity that needs **context**; that is exactly the gap neural segmenters, and subword tokenizers trained on Thai text, try to close.
- **Practical protocol**: sample 200 sentences from your own domain, segment with two engines, count disagreements, and read them. Disagreement rate is your error floor.
- Custom dictionaries (`pythainlp.util.Trie`) fix domain vocabulary: hospital names, drugs, procedures.

Cleaning steps that only exist for Thai

```
from pythainlp.util import normalize
normalize("แปลก  กิน  ๆ")   # typed with two เ
# -> "แปลก กิน ๆ"   (single แ, spaces collapsed)

from pythainlp.util import
thai_digit_to_arabic_digit
thai_digit_to_arabic_digit("๒๕๖๙")   # 2569
```

- **Look-alike characters**: เเ (two sara e) typed for แ; Thai and Arabic digits mixed; duplicated tone marks from keyboards.
- **Repetition mark** ๆ and abbreviation mark ฯ: decide whether to expand, keep, or drop, and do it consistently.
- **Zero-width characters** (U+200B, U+200C) inserted by web editors as line-break hints: invisible, but they split words for every tokenizer.
- **Buddhist-era years** (2569 = 2026) and Thai numerals inside dates: normalize before any date parser sees them.
- **Transliteration variants**: the same drug or hospital spelled several ways in Thai; canonicalize with a domain dictionary.

Each of these is a line in your datasheet: what you normalized, with which tool version, and why.

PART B1 · 2:00–2:20

From tokens to vectors: bag of words and tf-idf

The classical representation: a document is a vector of weighted counts. Still the fastest baseline you can build, still inside BM25 retrievers that feed modern RAG.

SOURCE DECK

Information Retrieval and RAG (November 2025), vector model and tf-idf sections, Speech and Language Processing, 3rd ed. draft · Dan Jurafsky and James H. Martin.

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors, posted freely for teaching · shown unmodified · selected pages

The vector space model of IR

Gerard Salton, 1971

Represent a document as a vector of counts of the words it contains.

Bag-of-words model

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

Vector representation of that doc

[1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1]

But raw frequency is a bad representation

- The co-occurrence matrices we have seen represent each cell by word frequencies.
- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- It's a paradox! How can we balance these two conflicting constraints?

Term frequency (tf) in the tf-idf algorithm

We could imagine using raw count:

$$\mathrm{tf}_{t,d} = \mathrm{count}(t,d)$$

But instead of using raw count, we usually squash a bit:

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document frequency (df)

df_t *is* the number of documents *t* occurs in.

(note this is not collection frequency: total count across all documents)

"*Romeo*" is very distinctive for one Shakespeare play:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

What is a document?

Could be a play or a Wikipedia article

But for the purposes of tf-idf, documents can be **anything**; we often call each paragraph a document!

Final tf-idf weighted value for a word

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Raw counts:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

tf-idf:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

TF-IDF nano-example

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

TF-IDF nano-example

Query: sweet love

$$|q| = \sqrt{(.125^2 + .301^2)} = .325$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	**1**	1	3	0.125	0.125	0.383
nurse	0					
love	**1**	1	2	0.301	0.301	0.924
how	0					
sorrow	0					
is	0					

TF-IDF nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{(.125^2 + .602^2)} = .615$$

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**	1	3	0.125	.125	.203	0.0779
nurse	0						
love	0						
how	0						
sorrow	**1**	1	1	.602	0.602	.979	0
is	0						

Final cosine

(details in chapter)

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

score(q,d1) = 0.747

score(q,d2) = 0.0779

- d1 has both terms, including 2 instances of *sweet*
- d2 is missing one of the terms

The nano example, vectorized

```
from sklearn.feature_extraction.text import (
    TfidfVectorizer)
docs = ["sweet sweet nurse love", "sweet sorrow",
        "how sweet is love", "nurse"]
tv = TfidfVectorizer(ngram_range=(1, 2),
                     stop_words="english")
X = tv.fit_transform(docs)      # shape (4, 9)
tv.get_feature_names_out()
# love, nurse, nurse love, sorrow, sweet,
# sweet love, sweet nurse, sweet sorrow, sweet
sweet

# at test time: transform, never fit_transform
X_test = tv.transform(new_docs)
```

- **fit on train, transform everything else**: the vocabulary and the idf weights are statistics of the training split. The image-class rule, verbatim.
- "how" and "is" disappeared: the stop-word list acted silently. **Every default is a decision.**
- Bigrams multiply features; `min_df` and `max_features` are your vocabulary budget, analogous to image size.
- For Thai, pass a `tokenizer=word_tokenize` callable: the vectorizer's default regex tokenizer assumes spaces.

PART B2 · 2:20–2:35

Classical text augmentation: EDA and back-translation

Flips and crops for text. Cheap, sometimes effective on small datasets, and a good way to see why "label-preserving" is harder for language than for images.

AUTHORED

Written for this course; operations follow **EDA** (Wei and Zou, EMNLP 2019) and back-translation (Sennrich et al., ACL 2016), run live with NLTK WordNet.

aclanthology.org/D19-1670 · aclanthology.org/P16-1009

Four operations on one clinical sentence (real outputs)

OPERATION	OUTPUT	VERDICT
original	`the patient reported mild chest pain after climbing stairs`	
synonym replacement (WordNet)	`the patient reported mild chest` **`ail`** `after` **`climb`** `stairs`	meaning kept, fluency lost
random swap	`the` **`stairs`** `reported mild chest pain after climbing` **`patient`**	nonsense, label maybe intact
random deletion (p = 0.15)	`patient reported mild pain after climbing stairs`	lost "chest": clinically different
random insertion	`the patient reported mild` **`very`** `chest pain after climbing stairs`	harmless noise

- EDA's own paper reports gains mostly for **small training sets** (hundreds of examples) with classical or RNN classifiers; gains shrink toward zero with pretrained transformers.
- Every operation risks the label. "Chest pain" without "chest" is another diagnosis. **Augment, then read a sample**, exactly as you plotted 8 augmented images.

Paraphrase by round trip, and when to bother

```
# back-translation: EN -> TH -> EN with any MT model
th = translate(en_text, src="en", tgt="th")
en2 = translate(th, src="th", tgt="en")
# "The patient reported mild chest pain after
#  climbing stairs" -> "The patient had slight chest
#  pain after going up the stairs"

# keep only paraphrases that stay close in meaning
if similarity(en_text, en2) > 0.8:
    augmented.append((en2, label))
```

Illustrative round trip; outputs depend on the MT system. Filter with a similarity model or a human read.

- Back-translation produces **fluent** paraphrases where EDA produces noise; the cost is an MT model per language pair and a filtering step.
- **Use when**: hundreds to a few thousand labelled examples, classical or fine-tuned small models, class imbalance in the minority class.
- **Skip when**: you fine-tune a large pretrained model on tens of thousands of examples; the model already knows the paraphrases.
- **Never** augment the validation or test set; evaluation sees original text only, the same rule as images.
- LLM-generated augmentation, and its collapse risk, is a class-2 topic.

PART C · 2:35–3:00

Clinical text, classical era: term spotting, negation, UMLS

Telegraphic notes, pertinent negatives, and thesauri with millions of concepts. The classical toolbox applied where every abbreviation matters.

SOURCE DECK

Lecture 7: Natural Language Processing, Part 1, 6.S897 / HST.956 Machine Learning for Healthcare, Spring 2019 · Peter Szolovits, MIT OpenCourseWare.
https://ocw.mit.edu/courses/6-s897-machine-learning-for-healthcare-spring-2019/

License: CC BY-NC-SA 4.0 · attribution page retained · third-party figures marked in the deck stay under their own terms









NLP



Massachusetts
Institute of
Technology

Outline

- **Value of the data in clinical text**
- Hyper-simplified linguistics
- Term spotting + handling negation, uncertainty
- ML to expand terms
- pre-NN ML to identify entities and relations
- language models
- Neural methods

Bulk of Valuable Data are in Narrative Text

orange=demographics
blue=patient condition, diseases, etc.
brown=procedures, tests
magenta=results of measurements
purple=time

Mr. Blind is a 79-year-old white white male with a history of diabetes mellitus, inferior myocardial infarction, who underwent open repair of his increased diverticulum November 13th at Sephsandpot Center.

The patient developed hematemesis November 15th and was intubated for respiratory distress. He was transferred to the Valtawnprinceel Community Memorial Hospital for endoscopy and esophagoscopy on the 16th of November which showed a 2 cm linear tear of the esophagus at 30 to 32 cm. The patient's hematocrit was stable and he was given no further intervention.

The patient attempted a gastrografin swallow on the 21st, but was unable to cooperate with probable aspiration. The patient also had been receiving generous intravenous hydration during the period for which he was NPO for his esophageal tear and intravenous Lasix for a question of pulmonary congestion.

On the morning of the 22nd the patient developed tachypnea with a chest X-ray showing a question of congestive heart failure. A medical consult was obtained at the Valtawnprinceel Community Memorial Hospital. The patient was given intravenous Lasix.

Telegraphic Language

3/11/98 IPN	(date of) Intern Progress Note,
SOB & DOE ↓	the patient's shortness of breath and dyspnea on exertion are decreased,
VSS, AF	the patient's vital signs are stable and the patient is afebrile,
CXR ⊕ LLL ASD no Δ	a recent new chest xray shows a left lower lobe air space density that is unchanged from the previous radiograph,
WBC 11K	a recent new white blood cell count is 11,000 cells per cubic milliliter,
S/B Cx ⊕ GPC c/w PC, no GNR	the patient's sputum and blood cultures are positive for gram positive cocci consistent with pneumococcus, no gram negative rods have grown,
D/C Cef →PCN IV	so the plan is to discontinue the cefazolin and then begin penicillin treatment intravenously.

Typical Goals of MNLP

- for any word or phrase, assign it a meaning (or null) from some taxonomy/ontology/ terminology;
  - e.g., “rheumatoid arthritis” ==> 714.0 (ICD9)
- for any word or phrase, determine whether it represents protected health information;
  - e.g., “Mr. Huntington suffers from Huntington’s Disease”
- determine aspects of each entity: time, location, certainty, ...
- having identified two meaningful phrases in a sentence, determine the relationship (or null) between them;
  - e.g., precedes, causes, treats, prevents, indicates, ...
  - note: we also need a taxonomy of relationships
- in a larger document, identify the sentences or fragments most relevant to answering a specific medical question;
  - e.g., where is the patient’s exercise regimen discussed?
- summarization
    - as data sets balloon in size, how to provide a meaningful overview

Two Types of Tasks

- Every word counts
  - De-identification
  - Extraction of all
    - entities
    - time
    - certainty
    - causation and association

- Aggregate judgment
  - E.g., “smoking” challenge
    - Most text may be irrelevant to specific result
  - Cohort selection—does a patient satisfy some set of inclusion and exclusion criteria
    - Often definite presence of a disease, complication, …

Term Spotting

- Traditionally, lists of coded items, narrative terms and patterns hand-crafted by researcher
- Negation and uncertainty handled by somewhat ad-hoc methods
  - NegEx is widely used, ∃ many more sophisticated variants
- Generalize terms
  - Manually or automatically identify high-certainty “anchors”
  - Learn related terms to augment the set of terms
    - From knowledge bases such as UMLS
    - From co-occurrence in EMR data
    - From co-occurrence in publications

Negation

- “Identifying pertinent negatives, then, involves identifying a proposition ascribing a clinical condition to a person and determining whether the proposition is denied or negated in the text.”
- Simpler than general problem of negation in NLP because negation applies mostly to noun phrases indicating diseases, tests, drugs, findings, …
- NegEx
  - Find all UMLS terms in each sentence of a discharge summary
    - “The patient denied experiencing chest pain on exertion” ⇒

      “The patient denied experiencing S1459038 on exertion”
  - Find patterns
    - <negation phrase> *{0,5} <UMLS term>
      - no signs of", "ruled out unlikely", "absence of", "not demonstrated", "denies", "no sign of", "no evidence of", "no", "denied", "without", "negative for", "not", "doubt", versus"
    - <UMLS term> *{0,5} <negation phrase>
      - “declined”, “unlikely”
  - Pseudo-negation: "gram negative", "no further", "not able to be", "not certain if", "not certain whether", not necessarily", "not rule out", "without any further", "without difficulty", "without further”

Chapman WW, Bridewell W, Hanbury P, Cooper GF, Buchanan BG. A simple algorithm for identifying negated findings and diseases in discharge summaries. J Biomed Inform. 2001 Oct;34(5):301-10.

NegEx results

- Baseline:
  - <negation phrase> * <UMLS term>
    - "no", "denies", "not", "without", "*n't", "ruled out", "denied"

	Baseline			NegEx		
	Group 1 sentences (i.e. containing NegEx negation phrases)	Group 2 sentences (i.e., not containing NegEx negation phrases)	All sentences	Group 1 sentences (i.e. containing NegEx negation phrases)	Group 2 sentences (i.e., not containing NegEx negation phrases)	All sentences
n	500	500	1000	500	500	1000
Sensitivity	88.27	0.00	**88.27**	82.31	0.00	77.84
Specificity	52.69	100.00	85.27	82.50	100.00	**94.51**
PPV	68.42	—	68.42	84.49	—	**84.49**
NPV	79.46	96.99	**93.01**	80.21	96.99	91.73

- Extremely simplistic schemes (kind of) work

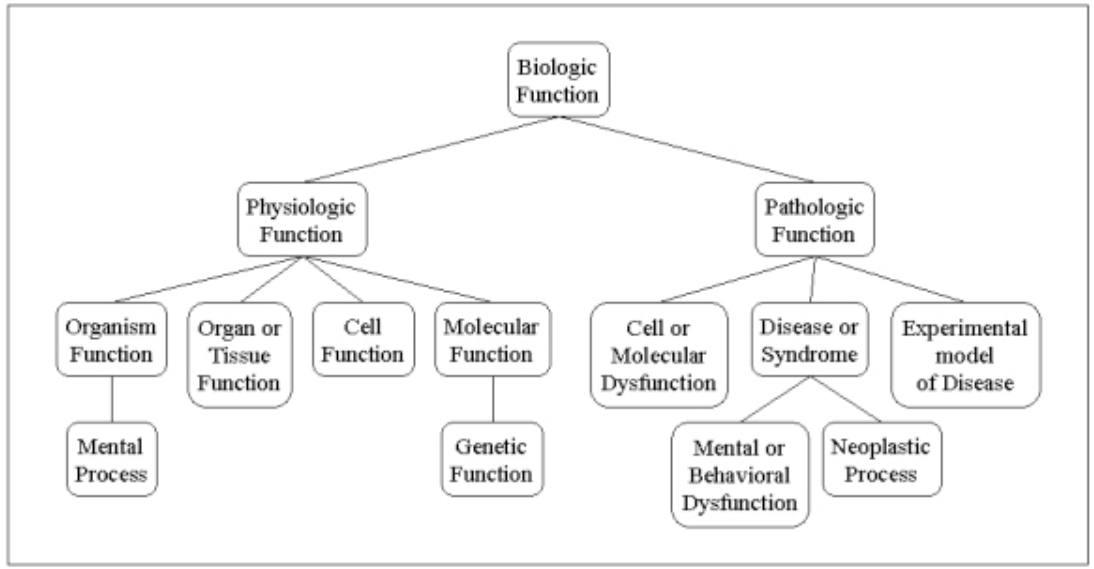
Generalize Terms

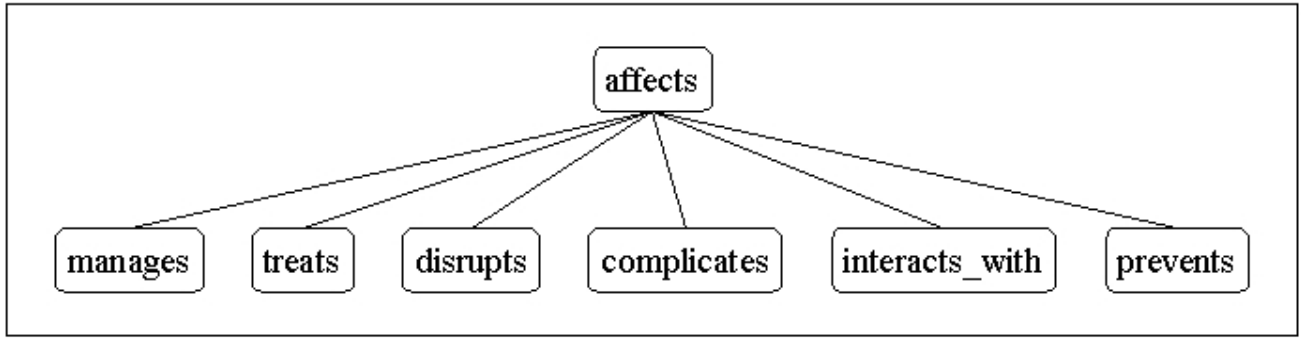
- Use synonymous terms as well as the starting ones
- Take advantage of others related terms
  - hypo- or hypernyms
  - other associated terms
    - e.g., common symptoms or treatments of a disease
- Recursive ML problem: learn how best to identify cases associated with a term
  - “phenotyping”

Available Classification Thesauri
Most Available through UMLS

- Unified Medical Language Systems project of NLM; since ~1985
- *Metathesaurus* now (2018ab version) includes 161 source vocabularies
  - MeSH, SNOMED, ICD-9, ICD-10, LOINC, RxNORM, CPT, GO, DXPLAIN, OMIM, ...
- Synonym mappings across vocabularies;
  - e.g., “heart attack” = “acute myocardial infarct” = “myocardial infarction” ...
  - 3,773,462 distinct concepts, represented by concept unique identifier (CUI)
- Jumbled compendium of every hierarchy drawn from every source
- *Semantic Network*
  - Hierarchy of
    - 54 relations
    - 127 types
  - Every CUI assigned ≥1 semantic type

Hierarchy of UMLS Semantic Network Types and Relations





Courtesy of National Library of Medicine. Image is in the public domain.

from http://www.ncbi.nlm.nih.gov/bookshelf/br.fcgi?book=nlmumls&part=ch05

Lexical Variant Generation (LVG) Tools

(from National Library of Medicine)

- Normalized words and phrases used as index to UMLS
- Lemmatization of words
  - stripping typical prefixes, suffixes
    - plurals, in-word negation, gerunds
- Discarding “noise” words, punctuation
- Lower-casing
- Alphabetic order of all remaining words

```
Mr. Huntington was admitted to Huntington Memorial Hospital for acute chest pain in March.
Mr. Huntington was admitted to Huntington Memorial Hospital for acute chest pain in March.|acute admit be chest hospital huntington huntington march
memorial mr pain
Mr. Huntington was admitted to Huntington Memorial Hospital for acute chest pain in March.|acute admit chest hospital huntington huntington march
memorial mr pain was
Mr. Huntington was admitted to Huntington Memorial Hospital for acute chest pain in March.|acute admitted be chest hospital huntington huntington march
memorial mr pain
Mr. Huntington was admitted to Huntington Memorial Hospital for acute chest pain in March.|acute admitted chest hospital huntington huntington march
memorial mr pain was

Weakness of the upper extremities
Weakness of the upper extremities|extremity upper weakness
```

MIT OpenCourseWare
https://ocw.mit.edu

6.S897 / HST.956 Machine Learning for Healthcare
Spring 2019

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Six regexes against a synthetic note

```
rules = [
  (r"\b\d{2}/\d{2}/\d{4}\b",        "[DATE]"),
  (r"\bMRN\s*\d+",                  "MRN [ID]"),
  (r"\b0\d{2}-\d{3}-\d{4}\b",       "[PHONE]"),
  (r"[\w.]+@[\w.]+\.\w+",           "[EMAIL]"),
  (r"\bDr\.\s+[A-Z]\w+",            "Dr. [NAME]"),
  (r"Pt:\s+[A-Z]\w+\s+[A-Z]\w+",    "Pt: [NAME]"),
]
for pat, rep in rules:
    note = re.sub(pat, rep, note)
```

in: Pt: Somchai Jaidee, MRN 4471923, DOB 12/03/1961, tel 081-234-5678. Seen 05/08/2026 at Srinagarind Hospital by Dr. Preecha; email s.jaidee@example.com.

out: Pt: [NAME], MRN [ID], DOB [DATE], tel [PHONE]. Seen [DATE] at Srinagarind Hospital by Dr. [NAME]; email [EMAIL].

- HIPAA Safe Harbor lists **18 identifier types**; regexes catch the structured ones (dates, phones, IDs, emails) and miss the unstructured ones (names inside prose, rare diseases, hospital names as here).
- Dates matter clinically: shift them consistently per patient instead of blanking, or you lose intervals.
- Rules are **auditable** and **fail loudly** on format changes; learned de-identifiers (class 2, OCW part 2) recall more and explain less. Production systems use both.
- Thai notes add Thai names, Thai-script dates, and ID numbers with different formats: the regex set is per language and per hospital.

What a Thai hospital note does to a classical pipeline

ผู้ป่วยชายอายุ 62 ปี U/D DM, HT มาด้วย chest pain 2 hr PTA, BP 150/90, no dyspnea, Dx: unstable angina, plan admit CCU

A typical (synthetic) telegraphic Thai note: Thai narrative, English abbreviations, numbers and units, no sentence punctuation.

- **Code switching inside a sentence**: the segmenter must switch between Thai maximal matching and English rules; a Thai-only dictionary shreds "dyspnea" into characters.
- **Abbreviations dominate** (U/D, HT, PTA, Dx, CCU): expansion tables are hospital-specific, and NegEx-style negation ("no dyspnea") works on the English fragments only if you keep them intact.
- **Stop-word lists and stemmers do not exist** for this mixture in any off-the-shelf form; build them from your corpus or skip them.
- De-identification needs **both** regex families (Thai ID formats, Buddhist-era dates) and English ones.
- All of this is **preprocessing you must document**: the same note, segmented differently, is a different dataset.

Before class 2

- **Task.** Take 200 sentences from your own project text. Run NFC normalization and count how many strings change. Tokenize with two tools (spaCy and NLTK, or two PyThaiNLP engines) and count disagreements. Bring the three most surprising disagreements.
- **Classifier check.** Build a tf-idf baseline with and without stop-word removal and stemming; report both scores. The gap is your evidence for or against classical preprocessing on your data.
- **Reading.** Jurafsky and Martin, SLP3 chapter 2 (free draft at `web.stanford.edu/~jurafsky/slp3`) · PyThaiNLP paper, NLP-OSS 2023 · Wei and Zou, EDA, EMNLP 2019.
- **Question to sleep on.** If two tokenizers produce different vocabularies from the same corpus, which one is "the data"?

NEXT TIME · CLASS 2

The LLM era: tokenizers, corpora, instructions, chunks

- HOUR 1 Subword tokenization: BPE from scratch, why language models see text the way they do, and the Thai tokenization premium measured live.
- HOUR 2 Corpus curation at web scale: Common Crawl, quality filters, deduplication, contamination, data mixing, and who is quietly excluded by "quality".
- HOUR 3 Instruction data and selection, RAG chunking, LLM-generated data and model collapse, clinical text with learned de-identification.

Bring the take-home disagreements: several of them will turn out to be tokenizer questions.

Source decks used in class 1

1 **Speech and Language Processing, 3rd edition draft, lecture slides**, Dan Jurafsky and James H. Martin. Decks used: Naive Bayes and Text Classification (January 2025), N-gram Language Models (January 2025), Words and Tokens (January 2026), Basic Text Processing (2023), Information Retrieval and RAG (November 2025).

https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors. Posted freely alongside the free book draft for teaching use; shown unmodified with attribution. Check the authors' terms before adapting or redistributing.

2 **MIT OpenCourseWare, 6.S897 / HST.956 Machine Learning for Healthcare, Spring 2019**, Peter Szolovits (lecturer), David Sontag (instructor). Lecture 7, Natural Language Processing, Part 1.

https://ocw.mit.edu/courses/6-s897-machine-learning-for-healthcare-spring-2019/

License CC BY-NC-SA 4.0; the deck's own OCW attribution page is retained. Figures marked "courtesy of" or "all rights reserved" inside the deck are excluded from the license and shown as published.

3 **PyThaiNLP** (Apache-2.0), Wannaphong Phatthiyaphaibun et al., "PyThaiNLP: Thai Natural Language Processing in Python", NLP-OSS 2023. Used live for the Thai segmentation demos.

https://pythainlp.org · https://aclanthology.org/2023.nlposs-1.4

Libraries, papers, and further reading

4 **Authored slides** (marked in the footer) were written for this course. Demos were executed with NLTK (Porter stemmer, WordNet lemmatizer, stop words, and the public-domain Gutenberg text of Austen's Emma for the Zipf, Heaps, and vocabulary counts), scikit-learn (TfidfVectorizer), ftfy, Python unicodedata, and PyThaiNLP; the clinical note and patient details are synthetic.

5 **Papers behind the authored slides**: Wei and Zou, EDA: Easy Data Augmentation Techniques, EMNLP 2019 · Sennrich, Haddow and Birch, Improving NMT with monolingual data (back-translation), ACL 2016 · Chapman et al., NegEx, J Biomed Inform 2001 · Gebru et al., Datasheets for Datasets, CACM 2021.

6 **Course cross-references**: section 2.1 (classical preprocessing and when it is harmful), 2.2 (subword tokenization, Thai tokenization premium), 2.9 (annotation), 2.10 (text augmentation), 2.12 (clinical text, i2b2/n2c2 de-identification, MIMIC).

7 **Compilation note.** Curated by T. Intharah for the Research Master's module Data Preparation for ML and AI, Khon Kaen University, September 2026. For classroom use; each source retains its own license.